

Assignment – 3 : Jack Compiler

Part – I : Jack Convolution Program

The program performs 2D convolution on an input matrix using a 3×3 filter. The implementation was written in Jack language and compiled into VM code using the custom Jack compiler. The program was tested using both 5×5 and 9×9 input matrices.

Exercise – 2

5×5 Input Matrix

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{bmatrix}$$

Filter Used (3×3)

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

This filter keeps only the center value from each 3×3 window.

Convolution Parameters

$$N = 5, \quad F = 3, \quad P = 0, \quad S = 1$$

Output Dimension Calculation

$$O = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

$$O = \left\lfloor \frac{5 - 3 + 0}{1} \right\rfloor + 1$$

$$O = 3$$

So, the output matrix size is:

$$3 \times 3$$

First Convolution Calculation

First 3×3 region:

$$\begin{bmatrix} 1 & 2 & 3 \\ 6 & 7 & 8 \\ 11 & 12 & 13 \end{bmatrix}$$

Applying the filter:

$$\begin{aligned} & (1 \times 0) + (2 \times 0) + (3 \times 0) \\ & +(6 \times 0) + (7 \times 1) + (8 \times 0) \\ & +(11 \times 0) + (12 \times 0) + (13 \times 0) \\ & = 7 \end{aligned}$$

Therefore,

$$Output[0][0] = 7$$

Final 5×5 Output

$$\begin{bmatrix} 7 & 8 & 9 \\ 12 & 13 & 14 \\ 17 & 18 & 19 \end{bmatrix}$$

This matches the VM execution output of the program.

Extra Credit – 9×9 Input Matrix

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ 10 & 11 & 12 & 13 & 14 & 15 & 16 & 17 & 18 \\ 19 & 20 & 21 & 22 & 23 & 24 & 25 & 26 & 27 \\ 28 & 29 & 30 & 31 & 32 & 33 & 34 & 35 & 36 \\ 37 & 38 & 39 & 40 & 41 & 42 & 43 & 44 & 45 \\ 46 & 47 & 48 & 49 & 50 & 51 & 52 & 53 & 54 \\ 55 & 56 & 57 & 58 & 59 & 60 & 61 & 62 & 63 \\ 64 & 65 & 66 & 67 & 68 & 69 & 70 & 71 & 72 \\ 73 & 74 & 75 & 76 & 77 & 78 & 79 & 80 & 81 \end{bmatrix}$$

Parameters

$$N = 9, \quad F = 3, \quad P = 0, \quad S = 1$$

Output Size Calculation

$$O = \left\lfloor \frac{9-3}{1} \right\rfloor + 1$$

$$O = 7$$

So the output size becomes:

$$7 \times 7$$

Final 9×9 Output

$$\begin{bmatrix} 11 & 12 & 13 & 14 & 15 & 16 & 17 \\ 20 & 21 & 22 & 23 & 24 & 25 & 26 \\ 29 & 30 & 31 & 32 & 33 & 34 & 35 \\ 38 & 39 & 40 & 41 & 42 & 43 & 44 \\ 47 & 48 & 49 & 50 & 51 & 52 & 53 \\ 56 & 57 & 58 & 59 & 60 & 61 & 62 \\ 65 & 66 & 67 & 68 & 69 & 70 & 71 \end{bmatrix}$$

Since the filter keeps only the center value, every output element becomes the center value of the corresponding 3×3 window.

Exercise – 3 : Analysis Questions

1) Tokenizer Design

The tokenizer removes comments before converting the program into tokens. Using only a simple regex on a single line is not reliable for handling multi-line comments.

Example:

```
/* This is  
a multi-line  
comment */
```

A single-line regex may only remove the first line and treat the remaining lines as normal code, which can produce invalid tokens and parser errors.

Another problem happens when comment symbols appear inside strings:

```
do Output.println("/* example */");
```

A simple regex may incorrectly remove part of the string.

To avoid these problems, the tokenizer uses a state-machine approach. It reads the file character by character and keeps track of whether it is inside:

- normal code,
- a single-line comment,
- a multi-line comment,
- or a string constant.

This makes tokenization more accurate and reliable.

2) Symbol Table State inside Conv.conv2d()

Inside the `conv2d()` method, the compiler stores all variables in the symbol table before generating VM code.

At the line:

```
let sum = sum + (inputVal * filterVal);
```

the following identifiers are in scope:

Identifier	Kind	Type	Index
ArrayFilter	field	Array	0
stride	field	int	1
ArrayInput	field	Array	2
ArrayOutput	field	Array	3
ArraySize	field	int	4
this	argument	Conv	0
outDim	local	int	0
i	local	int	1
j	local	int	2
fi	local	int	3
fj	local	int	4
sum	local	int	5
inRow	local	int	6
inCol	local	int	7
k	local	int	8
inputVal	local	int	9
filterVal	local	int	10

The compiler uses this table to decide:

- which VM segment to use,
- which memory index to access,
- and how VM commands should be generated.

For example, since `sum` is local variable index 5, the compiler accesses it using:

```
local 5
```

3) VM Code Trace for `ArrayOutput[k] = sum`

For the statement:

```
let ArrayOutput[k] = sum;
```

the compiler generates VM code similar to:

```

push this 3
push local 8
add

push local 5
pop temp 0

pop pointer 1

push temp 0
pop that 0

```

Explanation:

- `push this 3` pushes the base address of `ArrayOutput`.
- `push local 8` pushes the value of `k`.
- `add` computes the address of `ArrayOutput[k]`.

For the 5×5 test case:

- suppose `ArrayOutput` starts at RAM address 5000,
- and `k = 4`.

Then:

$$5000 + 4 = 5004$$

So `RAM[5004]` is the location of `ArrayOutput[4]`.

Next:

```
pop pointer 1
```

stores 5004 inside the `THAT` pointer.

Finally:

```
push temp 0
pop that 0
```

writes the value of `sum` into `RAM[5004]`.

This is called the `pointer 1 / that 0` array-access method in the Hack VM.

4) Effect of Convolution Stride

For the 9×9 input:

$$N = 9, \quad F = 3, \quad S = 2, \quad P = 0$$

Output dimension formula:

$$O = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

Substituting values:

$$O = \left\lfloor \frac{9 - 3}{2} \right\rfloor + 1$$

$$O = \lfloor 3 \rfloor + 1$$

$$O = 4$$

Therefore, the output size becomes:

$$4 \times 4$$

When stride is 1:

- the filter moves one cell at a time,
- more convolution windows are processed,
- and more VM instructions execute.

When stride is 2:

- the filter skips cells,
- fewer windows are processed,
- and execution becomes faster.

However, both versions still use nested loops over the matrix, so the overall asymptotic complexity remains approximately:

$$O(N^2)$$

The main difference is that stride 2 performs fewer total operations compared to stride 1.

Extra Credit – Debugging Experience

During testing, one major issue occurred while storing convolution results into the output array.

The problematic statement was:

```
let ArrayOutput[k] = sum;
```

Initially, the generated VM code did not correctly handle the array address before writing the value. Because of this, some output values were written to incorrect memory locations, which caused wrong convolution results.

The issue was identified by testing the smaller 5×5 matrix first. Since the expected output values were easy to calculate manually, it became easier to compare the actual output with the expected result and locate the bug.

After debugging the VM instructions step by step, the correct array assignment sequence was implemented using the THAT pointer mechanism.

The corrected VM logic performs the following operations:

- computes the exact address of `ArrayOutput[k]`,
- stores the address inside the THAT pointer,
- temporarily preserves the value of `sum`,
- and finally writes the value into the correct RAM location.

This debugging process improved understanding of:

- VM stack operations,
- pointer manipulation,
- array memory access,
- and low-level memory handling in the Hack platform.

The final implementation successfully generated the correct outputs for both the 5×5 and 9×9 convolution tests.